SIC and SIC/XE

수업 목표

- SIC/XE(Simplified Instructional Computer/Extra Expensive)의 개념에 대한 이해를 바탕으로 실제 관련 예제를 통해 이해도를 높인다.
- 다양한 SIC/XE 명령어 사용에 대해 실제 예제를 통해 이해도를 확인한다.

SIC Program Example (III-a)

- Looping and indexing operations

(STR2='TEST STRING'(STR1))

```
        LDX     ZERO            INITIALIZE INDEX REGISTER TO 0
MOVECH  LDCH    STR1,X          LOAD CHARACTER FROM STR1 INTO REG A
        STCH    STR2,X          STORE CHARACTER INTO STR2
        TIX     ELEVEN          ADD 1 TO INDEX, COMPARE RESULT TO 11
        JLT     MOVECH          LOOP IF INDEX IS LESS THAN 11
        .
        .
        .
STR1    BYTE    C'TEST STRING'  11-BYTE STRING CONSTANT
STR2    RESB    11              11-BYTE VARIABLE
.                               ONE-WORD CONSTANTS
ZERO    WORD    0
ELEVEN  WORD    11
```

LDCH m format(3/4) opcode(50) Effect: **A(rightmost byte] ← m**

TIX m format(3/4) opcode(2C) Effect: **X←(X)+1; (X): (m..m+2)**

Fig 1.4(a) Sample looping and indexing operations for SIC

SIC Program Example (III-b)

- Looping and indexing operations

  (STR2 = 'TEST STRING'(STR1))

TIXR 은 TIX와 비교해 값을 메로리로 부터 가져오는 것이 아니라 레지스터에 저장

```
        LDT    #11              INITIALIZE REGISTER T TO 11
        LDX    #0               INITIALIZE INDEX REGISTER TO 0
MOVECH  LDCH    STR1,X          LOAD CHARACTER FROM STR1 INTO REG A
        STCH    STR2,X          STORE CHARACTER INTO STR2
        TIXR    T               ADD 1 TO INDEX, COMPARE RESULT TO 11
        JLT     MOVECH          LOOP IF INDEX IS LESS THAN 11
        .
        .
        .
STR1    BYTE    C'TEST STRING'  11-BYTE STRING CONSTANT
STR2    RESB    11              11-BYTE VARIABLE
```

TIXR r1 format(2) opcode(B8) Effect: **X←(X)+1; (X):(r1)**

Fig 1.4(b) Sample looping and indexing operations for SIC/XE

SIC Program Example (IV-a)

- Indexing and looping operations

(GAMMA[I] = ALPHA[I] + BETA[I])

```
        LDA    ZERO        INITIALIZE INDEX VALUE TO 0
        STA    INDEX
ADDLP   LDX    INDEX       LOAD INDEX VALUE INTO REGISTER X
        LDA    ALPHA,X     LOAD WORD FROM ALPHA INTO REGISTER A
        ADD    BETA,X      ADD WORD FROM BETA
        STA    GAMMA,X     STORE THE RESULT IN A WORD IN GAMMA
        LDA    INDEX       ADD 3 TO INDEX VALUE
        ADD    THREE
        STA    INDEX
        COMP   K300        COMPARE NEW INDEX VALUE TO 300
        JLT    ADDLP       LOOP IF INDEX IS LESS THAN 300
        .
INDEX   RESW   1           ONE-WORD VARIABLE FOR INDEX VALUE
.                          ARRAY VARIABLES--100 WORDS EACH
ALPHA   RESW   100
BETA    RESW   100
GAMMA   RESW   100
.                          ONE-WORD CONSTANTS
ZERO    WORD   0
K300    WORD   300
THREE   WORD   3
```

Fig 1.5(a) Sample indexing and looping operations for SIC

SIC Program Example (IV-b)

- Indexing and looping operations
  (GAMMA[I] = ALPHA[I] + BETA[I])

```
         LDS    #3                 INITIALIZE REGISTER S TO 3
         LDT    #300               INITIALIZE REGISTER T TO 300
         LDX    #0                 INITIALIZE INDEX REGISTER TO 0
ADDLP    LDA     ALPHA,X           LOAD WORD FROM ALPHA INTO REGISTER A
         ADD     BETA,X            ADD WORD FROM BETA
         STA     GAMMA,X           STORE THE RESULT IN A WORD IN GAMMA
         ADDR    S,X               ADD 3 TO INDEX VALUE
         COMPR   X,T               COMPARE NEW INDEX VALUE TO 300
         JLT     ADDLP             LOOP IF INDEX VALUE IS LESS THAN 300
         .
         .
         .
.                                  ARRAY VARIABLES--100 WORDS EACH
ALPHA    RESW    100
BETA     RESW    100
GAMMA    RESW    100
```

COMPR r1, r2 format(2) opcode(A0) Effect: **(r1):(r2)**

Fig 1.5(b) Sample indexing and looping operations for SIC/XE

SIC Program Example (V)

- I/O operations

  (RECORD ← F1 device)

```
INLOOP  TD      INDEV           TEST INPUT DEVICE
        JEQ     INLOOP          LOOP UNTIL DEVICE IS READY
        RD      INDEV           READ ONE BYTE INTO REGISTER A
        STCH    DATA            STORE BYTE THAT WAS READ
        .
        .
```

LT(<):ready, **EQ(=)**:busy, **GT(>)**: not operational

```
OUTLP   TD      OUTDEV          TEST OUTPUT DEVICE
        JEQ     OUTLP           LOOP UNTIL DEVICE IS READY
        LDCH    DATA            LOAD DATA BYTE INTO REGISTER A
        WD      OUTDEV          WRITE ONE BYTE TO OUTPUT DEVICE
        .
        .
INDEV   BYTE    X'F1'           INPUT DEVICE NUMBER
OUTDEV  BYTE    X'05'           OUTPUT DEVICE NUMBER
DATA    RESB    1               ONE-BYTE VARIABLE
```

Fig 1.6 Sample input and output operations for SIC

SIC Program Example (VI-a)

- Subroutine call and record input operations

  (RECORD ← F1 device)

```
         JSUB    READ             CALL READ SUBROUTINE
         .
.                                 SUBROUTINE TO READ 100-BYTE RECORD
READ     LDX     ZERO             INITIALIZE INDEX REGISTER TO 0
RLOOP    TD      INDEV            TEST INPUT DEVICE
         JEQ     RLOOP            LOOP IF DEVICE IS BUSY
         RD      INDEV            READ ONE BYTE INTO REGISTER A
         STCH    RECORD,X         STORE DATA BYTE INTO RECORD
         TIX     K100             ADD 1 TO INDEX AND COMPARE TO 100
         JLT     RLOOP            LOOP IF INDEX IS LESS THAN 100
         RSUB                     EXIT FROM SUBROUTINE
         .
INDEV    BYTE    X'F1'            INPUT DEVICE NUMBER
RECORD   RESB    100              100-BYTE BUFFER FOR INPUT RECORD
.                                 ONE-WORD CONSTANTS
ZERO     WORD    0
K100     WORD    100
```

Fig 1.7(a) Sample subroutine call and record input operations for SIC

SIC Program Example (VI-b)

- Subroutine call and record input operations
  (RECORD ← F1 device)

```
          JSUB    READ                 CALL READ SUBROUTINE
           .
           .
.                                      SUBROUTINE TO READ 100-BYTE RECORD
READ      LDX    #0                    INITIALIZE INDEX REGISTER TO 0
          LDT    #100                  INITIALIZE REGISTER T TO 100
RLOOP     TD      INDEV                TEST INPUT DEVICE
          JEQ     RLOOP                LOOP IF DEVICE IS BUSY
          RD      INDEV                READ ONE BYTE INTO REIGSTER A
          STCH    RECORD,X             STORE DATA BYTE INTO RECORD
          TIXR    T                    ADD 1 TO INDEX AND COMPARE TO 100
          JLT     RLOOP                LOOP IF INDEX IS LESS THA 100
          RSUB                         EXIT FROM SUBROUTINE
           .
           .
INDEV     BYTE    X'F1'                INPUT DEVICE NUMBER
RECORD    RESB    100                  100-BYTE BUFFER FOR INPUT RECORD
```

JSUB m format(3/4) opcode(48) Effect: **L←(PC); PC←m**

Fig 1.7(b) Sample subroutine call and record input operations for SIC/XE

Classes of the Computer Architecture

- **CPU의 명령어 셋 아키텍쳐의 설계 방식**
  - CISC – Complex Instruction Set Computers
    - 복잡한 H/W 방식
    - VAX, Intel x86

  - RISC – Reduced Instruction Set Computers
    - 덜 복잡한 H/W 방식
    - UltraSPARC, PowerPC, Cray T3E
    - ARM(Aocrn RISC Machine → Advanced RISC Machine)

이 동영상 교과목 자료는

GJ-RIP 사업의

지자체-대학 협력기반 지역혁신 사업비로 개발되었음.